

DSC Machine Learning:

Unit 1

→ The fundamental goal in ML is to search through a space of functions & deduce the best mathematical relation that maps the given data to its expected outcomes.

→ A machine is said to learn if in doing a task T , its performance P improves with experience E .

* Notations:

- 1) Input Vector : $X \in \mathbb{R}^d$ → d -dimensional vectors that represents an instance observed. $X = [x_1, x_2, \dots, x_d]^T$.
- 2) Feature Space : $\mathcal{X} \subseteq \mathbb{R}^d$ → Vector Space containing all possible valid input representation.
- 3) Target : $y \in \mathcal{Y}$ → Outcome mapped to a input vector.
- 4) Datasets : $D = \{(x^{(i)}, y^{(i)})\}_{i=1}^N$ → Collection of N input observations mapped to its outcome, where all $x^{(i)}$ are drawn from IID's.
- 5) Model / Hypothesis : $h(x, \theta)$ or $\hat{y} = f(x)$ → A mathematical function that may fit the mapping of data inputs with outcome.
- 6) Model Space : $\mathcal{H}$ → Space of functions that can possibly best fit the $X \sim y$ mapping.
- 7) Loss Function : $\mathcal{E} = y - \hat{y}$ → Measures the error incurred in a single instance.

* Model Elements:

→ ML Model = Representation + Evaluation + Optimization

↓	↓	↓
Family of functions based on inductive bias which forms the hypothesis space $\mathcal{H}$	Quantitative metric to quantify the quality of a particular hypothesis function	Training / Search to get the best function with minimal loss.
Eg- Linear model, SVM, Decision Tree, Perceptrons etc.	Eg- MSE, MAE, Logloss, Infogain etc.	Eg- Gradient Descent etc.

DSC Machine Learning:

Unit 1

→ The fundamental goal in ML is to search through a space of functions & deduce the best mathematical relation that maps the given data to its expected outcomes.

→ A machine is said to learn if in doing a task T , its performance P improves with experience E .

* Notations:

- 1) Input Vector : $X \in \mathbb{R}^d$ → d -dimensional vectors that represents an instance observed. $X = [x_1, x_2, \dots, x_d]^T$.
- 2) Feature Space : $\mathcal{X} \subseteq \mathbb{R}^d$ → Vector space containing all possible valid input representation.
- 3) Target : $y \in Y$ → Outcome mapped to a input vector.
- 4) Datasets : $D = \{(x^{(i)}, y^{(i)})\}_{i=1}^N$ → Collection of N input observations mapped to its outcome, where all $x^{(i)}$ are drawn from IID's.
- 5) Model / Hypothesis : $h(x, \theta)$ or $\hat{y} = f(x)$ → A mathematical function that may fit the mapping of data inputs with outcome.
- 6) Model Space : $\mathcal{H}$ → Space of functions that can possibly best fit the $X \sim y$ mapping.
- 7) Loss Function : $\mathcal{E} = y - \hat{y}$ → Measures the error incurred in a single instance.

* Model Elements:

→ ML Model = Representation + Evaluation + Optimization

↓	↓	↓
Family of functions based on inductive bias which forms the hypothesis space $\mathcal{H}$	Quantitative metric to quantify the quality of a particular hypothesis function	Training / Search to get the best function with minimal loss.

Eg- Linear model, SVM, Decision Tree, Perceptrons etc.

Eg- MSE, MAE, Logloss, Infogain etc.

Eg- Gradient Descent etc.

① Supervised Learning:

— Here, the input dataset consists of input feature vectors with its corresponding label / outcome.

$$D = \{(x^{(1)}, y^{(1)})\}_{i=1}^N \rightarrow [x_1^{(1)}, x_2^{(1)}, \dots, x_d^{(1)}] \rightarrow y^{(1)}$$

$$[x_1^{(2)}, x_2^{(2)}, \dots, x_d^{(2)}] \rightarrow y^{(2)}$$

Supervised Learning

Regression

Target is continuous

$$y \in \mathbb{R}$$

$$e_i = y_i - \hat{y}_i$$

Classification

Target is categorical

$$y \in \{C_1, C_2, \dots, C_k\}$$

② Unsupervised Learning:

— Here, the input dataset consists of unlabeled input instances:

$$D = \{(x^{(1)})\}_{i=1}^N \rightarrow [x_1^{(1)}, x_2^{(1)}, \dots, x_d^{(1)}]$$

$$[x_1^{(2)}, x_2^{(2)}, \dots, x_d^{(2)}]$$

Unit 3: ① Linear Regression:

— Supervised learning algorithm that fits a linear relationship b/w independent feature variables x & a dependent target variable y .

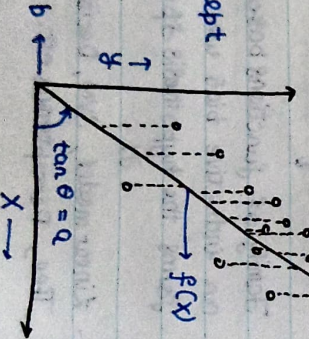
i.e. find the best straight line to model the relation b/w x & y .

$$\hat{y} = ax + b$$

Intercept

Predicted Independent

Target Slope Variable



① Simple Linear Regression: — Only one independent variable

— Model: $\hat{y} = ax + b$ — Model parameters = (a, b) — Model finds the best value

— Suppose given data, $D = \{(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)\}$

$$\text{Minimize, } e_i = y_i - \hat{y}_i \text{ where } \hat{y}_i = ax_i + b$$

$$e_i = y_i - (ax_i + b)$$

Loss function

— But this error funcⁿ is not suitable as e_i & $-e_i$ can cancel out.

— Loss function: $\hat{y}_i = ax_i + b$, $D = \{(x_i, y_i)\}_{i=1}^N$

$$e_i = (y_i - \hat{y}_i)$$

then,

$$J = \text{MSE} = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2 = \frac{1}{N} \sum_{i=1}^N (y_i - (ax_i + b))^2$$

∴ Goal — Find a, b such that hence, our goal reduces to a numerical optimization problem.

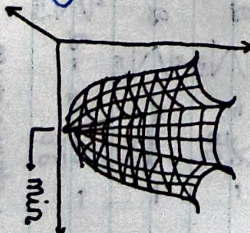
$$J = \frac{1}{N} \sum_{i=1}^N (y_i - (ax_i + b))^2 \text{ is minimum.}$$

— Directly finding a & b : (OLS)

$$\text{Objective funcⁿ: } \text{MSE}/J = \frac{1}{N} \sum_{i=1}^N (y_i - (ax_i + b))^2$$

$$\text{Goal, min } J(a, b)$$

Convex funcⁿ



— Find a and b where J reaches its minimum value

— Hence, we find a & b from:

$$\frac{\partial J}{\partial a} = 0 \text{ \& \& } \frac{\partial J}{\partial b} = 0$$

— As objective funcⁿ is a convex funcⁿ, we will find the global minimum at the stationary point.

Derivation:
$$L = \sum_{i=1}^n (\hat{y}_i - ax_i - b)^2$$

$\frac{1}{n}$ can be ignored, as it's a constant

$$-\frac{\partial L}{\partial a} = \sum_{i=1}^n \frac{\partial}{\partial a} (\hat{y}_i - ax_i - b)^2$$

$$-\frac{\partial L}{\partial a} = \sum_{i=1}^n 2(\hat{y}_i - ax_i - b) \left(\frac{\partial \hat{y}_i}{\partial a} - \frac{\partial ax_i}{\partial a} - \frac{\partial b}{\partial a} \right)$$

$$-\frac{\partial L}{\partial a} = \sum_{i=1}^n 2(\hat{y}_i - ax_i - b) (-x_i)$$

$$-\frac{\partial L}{\partial a} = -2 \sum_{i=1}^n x_i (\hat{y}_i - ax_i - b)$$

At minimum, $-2 \sum x_i (y_i - ax_i - b) = 0$
 $\Rightarrow \sum x_i (y_i - ax_i - b) = 0$

$$\Rightarrow \sum x_i y_i - a \sum x_i^2 - b \sum x_i = 0$$

$$\therefore \sum x_i y_i = a \sum x_i^2 + b \sum x_i \quad \text{--- (1)}$$

Now, $-\frac{\partial L}{\partial b} = \sum_{i=1}^n \frac{\partial}{\partial b} (y_i - ax_i - b)^2$

$$-\frac{\partial L}{\partial b} = \sum_{i=1}^n 2(y_i - ax_i - b) \left(\frac{\partial y_i}{\partial b} - \frac{\partial ax_i}{\partial b} - \frac{\partial b}{\partial b} \right)$$

$$-\frac{\partial L}{\partial b} = \sum_{i=1}^n 2(y_i - ax_i - b) (-1)$$

$$-\frac{\partial L}{\partial b} = -2 \sum_{i=1}^n (y_i - ax_i - b)$$

At minimum, $-2 \sum (y_i - ax_i - b) = 0$
 $\Rightarrow \sum y_i - a \sum x_i - \sum b = 0$

$$\Rightarrow \sum y_i - a \sum x_i - nb = 0$$

$$\therefore nb + a \sum x_i = \sum y_i$$

$$b = \frac{\sum y_i - a \sum x_i}{n} = \frac{\sum y_i}{n} - a \frac{\sum x_i}{n}$$

$$b = \bar{y} - a\bar{x} \quad \text{--- (2)}$$

From (1) & (2): $a \sum x_i^2 + b \sum x_i = \sum x_i y_i$
 $b = \bar{y} - a\bar{x}$

Substituting in (1): $a \sum x_i^2 + (\bar{y} - a\bar{x}) \sum x_i = \sum x_i y_i$

$$\Rightarrow a \sum x_i^2 + \bar{y} \sum x_i - a\bar{x} \sum x_i = \sum x_i y_i$$

$$\Rightarrow a \sum x_i^2 + n\bar{x}\bar{y} - a n\bar{x}^2 = \sum x_i y_i$$

$$\Rightarrow a \sum x_i^2 - a n\bar{x}^2 = \sum x_i y_i - n\bar{x}\bar{y}$$

$$\Rightarrow a = \frac{\sum x_i y_i - n\bar{x}\bar{y}}{\sum x_i^2 - n\bar{x}^2} \quad \text{--- (3)}$$

Multiply & divide by n ; $a = \frac{n \sum x_i y_i - \sum x_i \sum y_i}{n \sum x_i^2 - (\sum x_i)^2} = \frac{1/n \sum x_i y_i - \bar{x}\bar{y}}{\text{Var}(X)}$

Now, $\sum (x_i - \bar{x})(y_i - \bar{y})$

$$= \sum (x_i y_i - x_i \bar{y} - \bar{x} y_i + \bar{x} \bar{y})$$

$$= \sum x_i y_i - \sum x_i \bar{y} - \sum \bar{x} y_i + \sum \bar{x} \bar{y}$$

$$= \sum x_i y_i - n\bar{x}\bar{y} - n\bar{x}\bar{y} + \bar{x} n\bar{y}$$

$$= [\sum x_i y_i - n\bar{x}\bar{y}]$$

also, $\sum (x_i - \bar{x})^2$

$$= \sum (x_i^2 + \bar{x}^2 - 2x_i \bar{x})$$

$$= \sum x_i^2 + \sum \bar{x}^2 - 2\bar{x} \sum x_i$$

$$= \sum x_i^2 + n\bar{x}^2 - 2\bar{x} n\bar{x}$$

$$= [\sum x_i^2 - n\bar{x}^2]$$

$$\therefore a = \frac{\sum x_i y_i - n\bar{x}\bar{y}}{\sum x_i^2 - n\bar{x}^2} = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sum (x_i - \bar{x})^2}$$

thus, $b = \bar{y} - a\bar{x}$

Hence, $\frac{\partial L}{\partial a} = -2 \sum_{i=1}^n (y_i - ax_i - b) x_i$

$$\frac{\partial L}{\partial b} = -2 \sum_{i=1}^n (y_i - ax_i - b)$$

$$a = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sum (x_i - \bar{x})^2}$$

$$b = \bar{y} - a\bar{x}$$

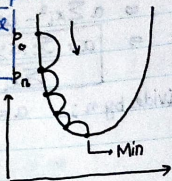
- Gradient Descent Method:

Our problem: $\left[\min_{a,b} \frac{1}{N} \sum_{i=1}^N (y_i - ax_i - b)^2 \right]$

Gradient Descent is an iterative optimization algorithm.

$$\begin{matrix} \text{New} \\ \text{parameter} \end{matrix} = \begin{matrix} \text{Old} \\ \text{parameter} \end{matrix} - \text{Learning Rate} \times \text{Gradient}$$

Go in the opposite direction of max loss in each step



$$\begin{aligned} a_{\text{new}} &= a_{\text{old}} - \eta G(x)_a \\ b_{\text{new}} &= b_{\text{old}} - \eta G(x)_b \end{aligned}$$

η = Learning Rate
Hyperparameter

Now, $\frac{\partial L}{\partial a} = -\frac{2}{N} \sum_{i=1}^N x_i (y_i - ax_i - b)^*$

$$\frac{\partial L}{\partial a} = -\frac{2}{N} \sum_{i=1}^N x_i (y_i - \hat{y}_i)$$

$$\frac{\partial L}{\partial b} = -\frac{2}{N} \sum_{i=1}^N (y_i - ax_i - b)$$

$$\frac{\partial L}{\partial b} = -\frac{2}{N} \sum_{i=1}^N (y_i - \hat{y}_i)$$

$$\nabla J = \begin{bmatrix} -\frac{2}{N} \sum_{i=1}^N (y_i - \hat{y}_i) \\ -\frac{2}{N} \sum_{i=1}^N x_i (y_i - \hat{y}_i) \end{bmatrix} \rightarrow \text{Gradient / Jacobian of } L$$

$$\begin{aligned} a_{\text{new}} &= a_{\text{old}} - \eta \frac{2}{N} \sum_{i=1}^N x_i (\hat{y}_i - y_i) \\ b_{\text{new}} &= b_{\text{old}} - \eta \frac{2}{N} \sum_{i=1}^N (\hat{y}_i - y_i) \end{aligned}$$

Gradient Descent Steps

Steps:

- 1- Randomly initialize a & b .
- 2- Predict $\hat{y}_i = ax_i + b \quad \forall x_i \in X$
- 3- Calculate $e_i = \hat{y}_i - y_i \quad \forall \hat{y}_i$ & y_i and $MSE = \frac{1}{N} \sum (y_i - \hat{y}_i)^2$
- 4- Calculate $\frac{\partial L}{\partial a} = \frac{2}{N} \sum x_i e_i$ & $\frac{\partial L}{\partial b} = \frac{2}{N} \sum e_i$
- 5- Update $a \leftarrow a - \eta \left(\frac{2}{N} \sum x_i e_i \right)$, $b \leftarrow b - \eta \left(\frac{2}{N} \sum e_i \right)$
- 6- Repeat until Δa or $\Delta b < \epsilon$ (Tolerance) or (Epoch)_{max} is reached. or ΔMSE

η : Learning Rate determines the size of the steps
If η is too small $\rightarrow$ algo moves slowly $\rightarrow$ Requires excess time
If η is too large $\rightarrow$ Too large jump $\rightarrow$ May diverge & skip the minimum

② Multiple Linear Regression:

Regression with multiple independent variables;

$$D = \{ [x_1^{(i)}, x_2^{(i)}, x_3^{(i)}, \dots, x_d^{(i)}], y^{(i)} \}_{i=1}^N \rightarrow d\text{-dimensional feature}$$

Feature Vector $\downarrow$ Target

Model:
$$\hat{y}_i = a_1 x_1^{(i)} + a_2 x_2^{(i)} + a_3 x_3^{(i)} + \dots + a_d x_d^{(i)} + b$$

$b \rightarrow$ intercept, $a_1, a_2, \dots, a_d \rightarrow$ Regression Coefficients

Eg-

$$\hat{y} \text{ (House price)} = a_1 (\text{Area}) + a_2 (\text{Bedrooms}) + a_3 (\text{Age}) + b$$

$\downarrow$
Predict House price

$\downarrow$
Features

$\downarrow$
Intercept

Coefficients represents the partial effects of each feature to predict the target variable.

Formulation:

Suppose we have n observations and p features.

Model:

$$\hat{y}_i = b + a_1 x_1^{(i)} + a_2 x_2^{(i)} + \dots + a_p x_p^{(i)}$$

actual value: y_i

hence,

$$e_i = y_i - \hat{y}_i$$

$$e_i = y_i - (b + a_1 x_1^{(i)} + a_2 x_2^{(i)} + \dots + a_p x_p^{(i)})$$

So,

$$MSE / \mathcal{L} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

$$\mathcal{L} = \frac{1}{n} \sum_{i=1}^n (y_i - (b + a_1 x_1^{(i)} + a_2 x_2^{(i)} + \dots + a_p x_p^{(i)}))^2$$

Goal: $\min_{b, a_1, a_2, \dots, a_p} (\mathcal{L})$ → Find best value of $b, a_1, a_2, \dots, a_p$ which gives minimum value of $\mathcal{L}$.

hence, to minimize we require:

$$\frac{\partial \mathcal{L}}{\partial b}, \frac{\partial \mathcal{L}}{\partial a_1}, \frac{\partial \mathcal{L}}{\partial a_2}, \dots, \frac{\partial \mathcal{L}}{\partial a_p} = 0 \rightarrow p+1 \text{ derivatives}$$

Now,

$$\frac{\partial \mathcal{L}}{\partial b} = \sum_{i=1}^n \frac{\partial}{\partial b} (y_i - b - \sum_{j=1}^p a_j x_j^{(i)})^2$$

$$\frac{\partial \mathcal{L}}{\partial b} = \sum_{i=1}^n \left(\frac{\partial y_i}{\partial b} - \frac{\partial b}{\partial b} - \sum_{j=1}^p \frac{\partial}{\partial b} a_j x_j^{(i)} \right) 2 (y_i - b - \sum_{j=1}^p a_j x_j^{(i)})$$

$$\frac{\partial \mathcal{L}}{\partial b} = -2 \sum_{i=1}^n \left(y_i - b - \sum_{j=1}^p a_j x_j^{(i)} \right)$$

To minimize, $-2 \sum_{i=1}^n (y_i - b - \sum_{j=1}^p a_j x_j^{(i)}) = 0$

$$\sum y_i - \sum b - \sum \sum a_j x_j^{(i)} = 0$$

$$\sum y_i = \sum b + \sum a_j x_j^{(i)}$$

$$\sum y_i = nb + a_1 \sum x_1^{(i)} + a_2 \sum x_2^{(i)} + \dots + a_p \sum x_p^{(i)}$$

Also,

$$X = \begin{bmatrix} 1 & x_1^{(1)} & x_2^{(1)} & \dots & x_p^{(1)} \\ 1 & x_1^{(2)} & x_2^{(2)} & \dots & x_p^{(2)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_1^{(n)} & x_2^{(n)} & \dots & x_p^{(n)} \end{bmatrix} \quad \beta = \begin{bmatrix} b \\ a_1 \\ a_2 \\ \vdots \\ a_p \end{bmatrix} \quad y = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}$$

Feature Matrix

$$\hat{y} = X\beta \quad \text{and} \quad e = y - \hat{y}$$

$$y = X\beta + e$$

Error Vector

Parameter Vector Target Vector

$$\text{here, } X\beta \rightarrow \begin{bmatrix} 1 & x_1^{(1)} & x_2^{(1)} & \dots & x_p^{(1)} \end{bmatrix} \cdot \begin{bmatrix} b \\ a_1 \\ a_2 \\ \vdots \\ a_p \end{bmatrix} = \begin{bmatrix} b + a_1 x_1^{(1)} + a_2 x_2^{(1)} + a_3 x_3^{(1)} + \dots + a_p x_p^{(1)} \end{bmatrix} = \hat{y}_i$$

Now,

$$\text{Error Loss} = (y - X\beta)^T (y - X\beta)$$

$$\mathcal{L} = y^T y - 2\beta^T X^T y + \beta^T X^T X \beta$$

$$\text{Now, } \frac{\partial \mathcal{L}}{\partial \beta} = -2X^T y + 2X^T X \beta \quad \text{Also, } \nabla \mathcal{L} = -2X^T (y - X\beta)$$

$$\text{At minimum, } -2X^T y + 2X^T X \beta = 0 \quad \therefore X^T y = X^T X \beta$$

$$X^T y = X^T X \beta$$

By gradient descent;

$$\beta \leftarrow \beta - \eta \nabla \mathcal{L}$$

$$\beta \leftarrow \beta - \eta \frac{2}{n} X^T (X\beta - y)$$

Normal Form OLS Solution

Directly gives value for all

coefficients & intercept

Gradient Descent Stop

⑤ Regression Evaluation Metrics:

$$1) \text{MSE} = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

$$2) \text{MAE} = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i|$$

$$3) \text{RMSE} = \sqrt{\frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2}$$

$$4) R^2 = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}$$

$$5) \text{Adj. } R = 1 - (1 - R^2) \frac{n-1}{n-p-1}$$

Unit 4 ① Logistic Regression:

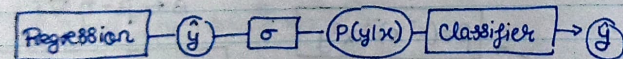
Logistic Regression models the probability of a binary outcome, resulting in classification.

Given a target $y = \{0, 1\}$ and a feature x .
we require to predict y when x is given

$$\therefore \text{we want: } \begin{bmatrix} P(Y=1|x) \\ P(Y=0|x) \end{bmatrix}$$

where, $0 \leq P(y|x) \leq 1$ Probability of a class

But linear regression does not necessarily generate $\hat{y}$ such that $0 \leq \hat{y} \leq 1$, hence, we require a function to convert $\hat{y} \rightarrow \{0, 1\}$



Logistic Regression

Sigmoid Function:

$$\sigma(z) = \frac{1}{1+e^{-z}}, \quad z \in (-\infty, \infty)$$

$$\& \sigma \in (0, 1)$$

$$\text{as } z \rightarrow -\infty, e^{-z} \rightarrow \infty, \sigma \rightarrow 0$$

$$z \rightarrow \infty, e^{-z} \rightarrow 0, \sigma \rightarrow 1$$

$\therefore$ Model:

$$P(Y=1|x) = \frac{1}{1+e^{-(ax+b)}}$$

$$\therefore [p = \sigma(ax+b) = \sigma(\hat{y})]$$

Linear Predictor / Logit score /
Log-Odds Score } z

Here,

b - Intercept: Determines position of curve at $x=0$

a - Coefficient: Determines behaviour of probability as x change

* Decision: If $p \geq 0.5 \Rightarrow \hat{y} = 1$ else if $p < 0.5 \Rightarrow \hat{y} = 0$

$\therefore$ when $z > 0, \sigma(z) \rightarrow p > 0.5 \Rightarrow \hat{y} = 1$

and if $z < 0, \sigma(z) \rightarrow p < 0.5 \Rightarrow \hat{y} = 0$

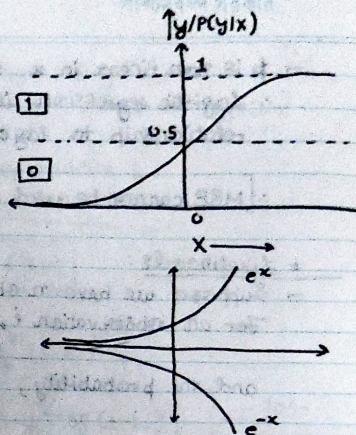
* Odds:
$$\text{Odds} = \frac{p}{1-p}$$

Now,
$$p = \frac{1}{1+e^{-z}}$$

$$1-p = 1 - \frac{1}{1+e^{-z}} = \frac{1+e^{-z}-1}{1+e^{-z}} = \frac{e^{-z}}{1+e^{-z}}$$

$$\therefore \text{Odds} = \frac{1}{1+e^{-z}} \cdot \frac{1+e^{-z}}{e^{-z}} = \frac{1}{e^{-z}} = e^z$$

also, $z = ax+b$



$$\therefore \text{Odds} = \frac{p}{1-p} = e^{ax+b}$$

$$\& \log\left(\frac{p}{1-p}\right) = ax+b$$

Log-Odds /
Logit

$\rightarrow$ Odds is the ratio that an event will happen to it will not happen

→ p is non linear in x BUT log odds is linear in x
 ∴ Logistic regression is a model which gives a linear relationship to log odds/logit

∴ [MSE cannot be used as a loss function]

* Likelihoods:

→ Suppose we have n observations,

For an observation i ,

$$y_i \in \{0, 1\}$$

and the probability, $p_i = P(Y_i = 1 | x_i)$

$$p_i = \sigma(ax_i + b)$$

If $y_i = 1$

then probability of getting actual result = p_i

If $y_i = 0$

then probability of getting actual result = $1 - p_i$

then $P(Y_i = y_i) = \frac{y_i}{p_i} (1 - p_i)^{1 - y_i}$ $y_i \rightarrow \text{class}$

then,

$$P(Y_i = y_i) = p_i^{y_i} (1 - p_i)^{1 - y_i} \quad y_i \rightarrow \text{class}$$

If $y_i = 1$ $P(Y_i = 1) = p_i^1 (1 - p_i)^0 = p_i$

If $y_i = 0$ $P(Y_i = 0) = p_i^0 (1 - p_i)^1 = 1 - p_i$

For the entire dataset,

$$L(a, b) = \prod_{i=1}^n p_i^{y_i} (1 - p_i)^{1 - y_i}$$

Goal, find a, b to maximize likelihood $L(a, b)$

$$\max_{a, b} L(a, b)$$

[Eg- If $p_i = 60\%$ → 60% chances it belongs to $y_i = 1$
 & $1 - 60 = 40\%$ chances it belongs to $y_i = 0$]

Hence, we find a, b such that

$$p_i \rightarrow 1 \text{ when } y_i = 1 \quad \therefore p_i \text{ approaches } 1$$

$$\& p_i \rightarrow 0 \text{ when } y_i = 0 \quad \therefore p_i \text{ approaches } 0.$$

We maximize this likelihood

$$\max_{a, b} L(a, b)$$

→ Taking log of L ; $\log L = \log \left[\prod_{i=1}^n p_i^{y_i} (1 - p_i)^{1 - y_i} \right]$

$$\left[\log L = \sum_{i=1}^n \left[y_i \log p_i + (1 - y_i) \log (1 - p_i) \right] \right] \rightarrow \text{Log-likelihood}$$

We prefer minimizing loss rather than maximize;

$$\log L = -\frac{1}{N} \sum_{i=1}^N [y_i \log p_i + (1 - y_i) \log (1 - p_i)] \rightarrow \text{Log Loss}$$

∴ Objective Goal: $\min_{a, b} \log L$ → Find a, b to minimize log loss

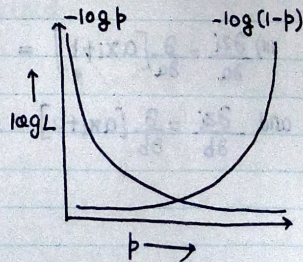
when $p \rightarrow 1$ as $y = 1$ $-\log(p) \rightarrow 0 \Rightarrow$ Very less loss

$p \rightarrow 0$ as $y = 1$ $-\log(p) \rightarrow 1 \Rightarrow$ High loss → High penalty

→ Training by Gradient Descent:

$$\text{we know, } \theta_{\text{new}} = \theta_{\text{old}} - \eta \nabla J$$

where $J = \log L$



$$J_i \rightarrow p_i \rightarrow z_i$$

$$\text{Now, } J = -\frac{1}{N} \sum [y_i \log p_i + (1-y_i) \log (1-p_i)]$$

$$\Rightarrow \frac{\partial J}{\partial a} \rightarrow \frac{\partial J}{\partial a} = \frac{\partial J}{\partial p_i} \frac{\partial p_i}{\partial z_i} \frac{\partial z_i}{\partial a}$$

$$\text{then } \frac{\partial J_i}{\partial a} = \frac{\partial J_i}{\partial p_i} \cdot \frac{\partial p_i}{\partial z_i} \cdot \frac{\partial z_i}{\partial a}$$

$$\frac{\partial J_i}{\partial p_i} = \frac{\partial}{\partial p_i} [-(y_i \log p_i + (1-y_i) \log (1-p_i))]$$

$$\frac{\partial J_i}{\partial p_i} = -\left[y_i \left(\frac{1}{p_i}\right) + (1-y_i) \left(\frac{1}{1-p_i}\right) (-1)\right]$$

$$\frac{\partial J_i}{\partial p_i} = -\left[\frac{y_i}{p_i} - \frac{1-y_i}{1-p_i}\right]$$

$$\frac{\partial J_i}{\partial p_i} = -\frac{y_i}{p_i} + \frac{1-y_i}{1-p_i}$$

$$\text{ii) } \frac{\partial p_i}{\partial z_i} = \frac{\partial}{\partial z_i} \left(\frac{1}{1+e^{-z_i}}\right) = \frac{\partial}{\partial z_i} \left(\frac{1}{e^{z_i}+1} \times e^{z_i}\right) = \frac{\partial}{\partial z_i} \left(\frac{e^{z_i}}{1+e^{z_i}}\right)$$

$$= \frac{1+e^{z_i} - e^{z_i}}{(1+e^{z_i})^2} = \frac{1}{1+e^{z_i} + e^{z_i}}$$

$$\frac{\partial p_i}{\partial z_i} = \sigma(z) (1-\sigma(z))$$

$$\therefore \frac{\partial p_i}{\partial z_i} = p_i (1-p_i)$$

$$\text{iii) } \frac{\partial z_i}{\partial a} = \frac{\partial}{\partial a} [ax_i + b] = x_i$$

$$\text{and } \frac{\partial z_i}{\partial b} = \frac{\partial}{\partial b} [ax_i + b] = 1$$

$$\text{Hence, } \frac{\partial J_i}{\partial a} = \left(-\frac{y_i}{p_i} + \frac{1-y_i}{1-p_i}\right) p_i (1-p_i) \cdot x_i$$

$$= \frac{-y_i(1-p_i) + p_i(1-y_i)}{p_i(1-p_i)} \times p_i(1-p_i) \times x_i$$

$$= -y_i + p_i + p_i - y_i p_i = (-y_i + y_i p_i + p_i - y_i p_i) x_i$$

$$\frac{\partial J_i}{\partial a} = (p_i - y_i) x_i$$

$$\frac{\partial J_i}{\partial b} = (p_i - y_i)$$

$\frac{1}{n}$ ignored
was added

Descent steps:

$$a \leftarrow a - \eta \frac{x_i(p_i - y_i)}{n}$$

$$b \leftarrow b - \eta \frac{(p_i - y_i)}{n}$$

Steps:

- 1- Randomly initialize a & b .
- 2- Calculate linear parameter $\rightarrow z_i = ax_i + b$
- 3- Apply Sigmoid $\rightarrow \sigma(z_i) = \frac{1}{1+e^{-z_i}} \rightarrow p_i$
- 4- Calculate loss $\rightarrow J(a, b) = -\frac{1}{N} \sum_{i=1}^N [y_i \log p_i + (1-y_i) \log (1-p_i)]$
- 5- Calculate gradient $\rightarrow \frac{\partial J}{\partial a} = \frac{1}{N} \sum x_i (p_i - y_i)$ & $\frac{\partial J}{\partial b} = \frac{1}{N} \sum (p_i - y_i)$
- 6- Update parameter $\Rightarrow a \leftarrow a - \eta \frac{\partial J}{\partial a}$ & $b \leftarrow b - \eta \frac{\partial J}{\partial b}$
- 7- Repeat until Δa & $\Delta b < \epsilon$, epochs reached or ΔLogL

* Decision Boundary:

when $z=0$, $p=0.5$

Here, $ax + b = 0$

$$\Rightarrow x = -b/a \rightarrow \text{Decision Boundary}$$

③ k-Nearest Neighbours:

→ A lazy supervised learning algorithm that predicts the class of a given feature vector by looking at its closest neighbours.

∴ KNN = Look at k closest neighbour & use their target values

→ KNN is lazy learner becoz:

KNN stores the training data & computes closest neighbours only when the new observation is given.

Steps: Suppose we have n observations

For each observation we have p features:

$$X_i = [x_{i1}^{(1)}, x_{i2}^{(1)}, \dots, x_{ip}^{(1)}] \text{ \& } y_i = \text{target}$$

Now,

- New observation $X^* = [x_1^*, x_2^*, \dots, x_p^*]$
- Calculate distance d between X^* and all $X_i \in X$
- Sort in ascending order
- Pick top k neighbours
- Use majority voting to predict y^*

$$\hat{y}_* = \text{mode} \{y_i : i \in \text{Nearest Neighbours}(X^*)\}$$

For regression:

$$\hat{y}_* = \frac{1}{k} \sum_{i \in N(X^*)} y_i$$

Here,

$$d = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2} \quad \leftarrow \text{Euclidean}$$

$$d = |x_1 - x_2| + |y_1 - y_2| \quad \leftarrow \text{Manhattan}$$

$$d = \left(\sum_{j=1}^p |x_j - y_j|^q \right)^{1/q} \quad \leftarrow \text{Minkowski}$$

→ KNN is sensitive to the scale of variables, hence, all variables must be feature scaled before applying KNN.

For scaling:

$$\bar{x} = \frac{x - \mu}{\sigma}$$

or

$$x = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

Steps: 1- Given $D = \{(x_i, y_i)\}_{i=1}^N$

2- Given new point X_*

3- Choose k & distance metric d

4- Calculate $d_i = d(X_i, X_*) \quad \forall i \in [1, N]$

5- Sort $d_{(1)} \leq d_{(2)} \leq \dots \leq d_{(n)}$

6- Choose top k values $N_k(X^*)$

7- For classification:

$$\hat{y} = \text{mode} \{y_i : i \in N_k(X^*)\}$$

8- For Regression:

$$\hat{y} = \frac{1}{N_k} \sum_{i \in N(X^*)} y_i$$

③ Naïve-Bayes Classifier:

→ Probabilistic classification algorithm.

→ For each class C, and given features X, it calculates $P(C|x) \quad \forall i$ then chooses the class with highest probability.

* Probability Background:

$$\rightarrow P(A|B) = \frac{P(A \cap B)}{P(B)} \quad \leftarrow \text{Conditional probability}$$

$$\therefore P(A \cap B) = P(A|B) P(B)$$

$$= P(B|A) P(A)$$

$$\therefore P(A|B) P(B) = P(B|A) P(A)$$

$$P(A|B) = \frac{P(B|A) P(A)}{P(B)}$$

Bayes Theorem

Bayes Theorem:
$$P(Y|X) = \frac{P(X|Y) P(Y)}{P(X)}$$

Likelihood
Prior

↓
↑

Posterior
Total / Evidence

Naive Bayes Assumption:
Features are conditionally independent in a given class.

Suppose $X = [x_1, x_2, \dots, x_p]$ & class $Y = C_k$
We need to find:

$$P(Y = C_k | X) = P(Y = C_k | [x_1, x_2, \dots, x_p])$$

Now,

By Bayes Theorem;
$$P(Y = C_k | X) = \frac{P(X | Y = C_k) P(Y = C_k)}{P(X)}$$

Now, by Naive Bayes Assumptions;

$$\begin{aligned} P(X | Y = C_k) &= P([x_1, x_2, \dots, x_p] | Y = C_k) \\ &= \prod_{i=1}^p P(x_i | Y = C_k) \end{aligned}$$

$$\therefore P(Y = C_k | X) = \frac{\prod_{i=1}^p P(x_i | C_k) \cdot P(C_k)}{P(X)}$$

$$\Rightarrow P(Y = C_k | X) \propto \prod_{i=1}^p P(x_i | C_k) \cdot P(C_k)$$

Hence,

$$\hat{y} = \arg \max_k \left[P(C_k) \cdot \prod_{i=1}^p P(x_i | C_k) \right]$$